

PRÁTICAS DE PROGRAMAÇÃO · AULA 04

Arquivos & Gerenciamento de módulos

Capítulos 7 e 8 · Práticas de Programação · Editora Senac SP

Prof. Rafael Pereira

Onde estamos na jornada

O que já construímos — e o que ainda falta ao estoque

Aulas 01–02

Funções · listas · dicionários



Aula 03

Lambda · exceções



Oficina

PTI: estoque em memória



Aula 04

Arquivos · módulos

Dois problemas que o PTI deixou

1. Fecha o programa → o estoque some. Cadastrou 40 produtos? Perdeu.
2. `ler_inteiro_nao_negativo` está colada dentro do estoque. Outro programa precisa? Copia e cola.

Duas respostas de hoje

1. Arquivos (cap. 7): `open`, `with`, `read`, `write`, `pickle`.
2. Módulos (cap. 8): `import validacao` — e a função passa a existir em qualquer programa da pasta.

Lembrete: no PTI oficial, R8 exige dados só em memória. O que fazemos hoje é a versão 2 — para aprender, não para entregar.

open(): a porta para o disco

Arquivo = sequência de caracteres num nome. O modo diz o que você vai fazer com ele.

```
open("notas.txt", "w", encoding="utf-8")
```

nome (relativo à pasta aberta no VS Code) · modo · encoding: sempre utf-8 (acentos!)

Modo	Faz o quê	Se o arquivo não existe	Se já existe
"r"	ler (é o padrão)	FileNotFoundError	lê do início
"w"	escrever do zero	cria	APAGA tudo e recomeça
"a"	acrescentar no fim	cria	mantém e escreve depois
"x"	criar, e só criar	cria	FileExistsError
"rb" / "wb"	mesma coisa, em bytes	idem	idem — sem encoding

Regra de ouro: escolha o modo pensando no que acontece se o arquivo JÁ existir. "w" é o que mais apaga trabalho por acidente.

with: abre, usa e fecha sozinho

Sem close(), o conteúdo pode nem chegar ao disco

```
# jeito manual: você é responsável pelo close()
arquivo = open("notas.txt", "w", encoding="utf-8")
n = arquivo.write("Ana;8.5\n")
print(n)          # write devolve nº de caracteres
8
arquivo.write("Bruno;6.0\n")
arquivo.close()   # esqueceu? dados podem ficar no buffer
```

```
# jeito certo: with fecha ao sair do bloco
with open("notas.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write("Ana;8.5\n")
    arquivo.write("Bruno;6.0\n")
print(arquivo.closed)
True
```

Por que fechar importa

O Python guarda o que você escreve num buffer e só grava no disco de vez em quando. close() força a gravação e libera o arquivo para outros programas.

Depois do with, acabou

```
arquivo.read()
ValueError: I/O operation on
closed file.
```

Quer ler de novo? Abre de novo.

\n dentro da string = quebra de linha no arquivo. Sem ele, write("a") e write("b") viram "ab" na mesma linha.

Quatro jeitos de ler

`read()` · `readline()` · `readlines()` · `for linha in arquivo`

```
with open("notas.txt", encoding="utf-8") as a:
    conteudo = a.read()          # tudo, numa str
    print(repr(conteudo))
    'Ana;8.5\nBruno;6.0\n'

with open("notas.txt", encoding="utf-8") as a:
    print(repr(a.readline()))    # uma linha por vez
    print(repr(a.readline()))
    print(repr(a.readline()))
    'Ana;8.5\n'
    'Bruno;6.0\n'
    ''                          # acabou: vazio, não erro

with open("notas.txt", encoding="utf-8") as a:
    print(a.readlines())         # lista de linhas
    ['Ana;8.5\n', 'Bruno;6.0\n']
```

```
# o jeito mais usado
with open("notas.txt",
encoding="utf-8") as a:
    for linha in a:
        print(linha.strip())
Ana;8.5
Bruno;6.0
```

O \n vem junto

Cada linha lida termina em `\n`. `print()` põe outro → linha em branco entre cada uma. `strip()` resolve. É o mesmo `strip()` da aula 02.

Arquivo texto devolve sempre str. "8.5" é texto até você fazer `float("8.5")` — igual ao `input()` da aula 03.

Quatro armadilhas (todas executadas)

Cada uma vai aparecer em algum monitor da sala hoje

1. "w" apaga só de abrir

```
with open("copia.txt", "w") as a:
    pass          # não escreveu nada
print(open("copia.txt").read())
''               # e o conteúdo já era
```

2. write() só aceita str

```
a.write(10)
TypeError: write() argument
must be str, not int

a.write(str(10))    ou    f"{10}\n"
```

3. read() duas vezes = vazio

```
print(len(a.read()))    # 28
print(repr(a.read()))   # ''
a.seek(0)               # volta ao início
print(len(a.read()))    # 28
```

4. Sem encoding, acento quebra

```
gravou "Ação" em utf-8, leu em latin-1:
AÃ$Ãfo

encoding="utf-8" nos dois lados. Sempre.
```

A 3 é o iterador que esgota — o mesmo comportamento do map() da aula 03. Cursor no fim = nada para ler.

notas.txt: gravar, ler e calcular a média

```
# 1) gravar 3 alunos, um por linha, separados por ;  
with open("notas.txt", "w", encoding="utf-8") as arquivo:  
    arquivo.write("Ana;8.5\n")  
    arquivo.write("Bruno;6.0\n")  
    arquivo.write("Carla;9.0\n")
```

```
# 2) ler, separar, converter, somar  
soma = 0  
quantidade = 0  
with open("notas.txt", encoding="utf-8") as arquivo:  
    for linha in arquivo:  
        nome, nota = linha.strip().split(";")  
        soma += float(nota)  
        quantidade += 1  
        print(f"{nome}: {float(nota):.1f}")  
print(f"Média da turma: {soma / quantidade:.2f}")  
Ana: 8.5  
Bruno: 6.0  
Carla: 9.0  
Média da turma: 7.83
```

Como funciona

`strip()` tira o `\n`. `split(";")` devolve 2 strings. `float()` converte. Sem o `float`, `"8.5" + 0` dá `TypeError`.

Critério de conclusão

✓ notas.txt aparece no explorador do VS Code com 3 linhas.

✓ Média 7.83.

Extra: abra em "a" e acrescente você. A média muda?

Arquivo que não existe é exceção

Cap. 6 + Cap. 7: o programa começa do zero em vez de morrer

```
try:
    with open("dados.txt", encoding="utf-8") as arquivo:
        print(arquivo.read())
except FileNotFoundError:
    print("Arquivo não encontrado. Começando do zero.")
Arquivo não encontrado. Começando do zero.
```

```
# modo "x": cria e recusa sobrescrever
open("novo.txt", "x").close()
open("novo.txt", "x")
FileExistsError: [Errno 17] File exists: 'novo.txt'
```

O with dentro do try

Se o open() falhar, o with nem começa e cai no except. Se der certo, o with fecha e o except é ignorado.

Ponte para a prática 2

Na 1ª execução não existe estoque.pkl. O programa não pode travar: devolve [] e segue. É exatamente este try.

Quadro de bolso da aula 03 ganhou duas linhas: FileNotFoundError (abrir em "r" o que não existe) e FileExistsError (abrir em "x" o que existe).

Arquivo binário: bytes em vez de str

"wb" / "rb" — e por que str não entra

```
with open("b.bin", "wb") as arquivo:
    arquivo.write(b"ola")                # literal bytes
    arquivo.write("ação".encode("utf-8")) # str -> bytes

with open("b.bin", "rb") as arquivo:
    dados = arquivo.read()
print(dados)
b'olaa\xc3\xa7\xc3\xa3o'
print(type(dados))
<class 'bytes'>
print(dados.decode("utf-8"))           # bytes -> str
olaação
```

Erros para reconhecer

a.write("ola") em "wb"
TypeError: a bytes-like object is required, not 'str'

open(..., "rb", encoding="utf-8")
ValueError: binary mode doesn't take an encoding argument

Texto × binário

Texto: o Python traduz bytes ↔ caracteres pelo encoding. Binário: entrega os bytes crus. Imagens, áudio, e objetos Python (pickle) são binário.

ç em utf-8 são 2 bytes (\xc3\xa7). Por isso 'Ação' tem 4 letras e 6 bytes — e por isso o encoding importa.

pickle: guarda o objeto Python inteiro

str(lista) grava um desenho da lista. pickle grava a lista.

```
# o problema: texto volta como texto
estoque = [{"codigo": 101, "nome": "Teclado", "preco": 79.9, "quantidade": 10}]
with open("t.txt", "w", encoding="utf-8") as a:
    a.write(str(estoque))
lido = open("t.txt", encoding="utf-8").read()
print(type(lido), lido[0])
<class 'str'> [          # lido[0] é a letra "[", não o dicionário
```

```
import pickle
with open("estoque.pkl", "wb") as a:
    pickle.dump(estoque, a)          # objeto -> bytes -> disco
with open("estoque.pkl", "rb") as a:
    carregado = pickle.load(a)       # disco -> bytes -> objeto
print(carregado == estoque, carregado is estoque)
True False                          # mesmo conteúdo, cópia nova
print(type(carregado[0]["preco"]))
<class 'float'>                     # os tipos voltam certos
```

Erros para reconhecer

dump em modo "w":
TypeError: write() argument
must be str, not bytes

load de arquivo vazio:
EOFError: Ran out of input

dump = despejar; load = carregar. Só abra .pkl que VOCÊ gerou: pickle executa código ao carregar, e um arquivo de terceiros pode ser malicioso.

salvar_estoque / carregar_estoque

```
import pickle

def salvar_estoque(estoque, nome_arquivo="estoque.pkl"):
    """Grava a lista de dicionários em arquivo binário."""
    with open(nome_arquivo, "wb") as arquivo:
        pickle.dump(estoque, arquivo)

def carregar_estoque(nome_arquivo="estoque.pkl"):
    """Lê a lista do arquivo; se ele não existir, começa vazia."""
    try:
        with open(nome_arquivo, "rb") as arquivo:
            return pickle.load(arquivo)
    except FileNotFoundError:
        return []

print(carregar_estoque())          # sem estoque.pkl na pasta
[]
salvar_estoque([{"codigo": 1, "nome": "Cabo",
                  "preco": 25.5, "quantidade": 7}])
print(carregar_estoque())
[{'codigo': 1, 'nome': 'Cabo', 'preco': 25.5, 'quantidade': 7}]
```

Onde encaixa no PTI

```
main(): estoque =
carregar_estoque()

cadastrar_produto():
salvar_estoque(estoque)
logo após o append.
```

Critério de conclusão

- ✓ 1º print: []
- ✓ estoque.pkl aparece na pasta.
- ✓ 2º print: a lista volta.
- ✓ Feche o Python, rode só carregar → ainda volta.

Módulo = um arquivo .py que você importa

Cap. 8 — você já usa módulos desde a aula 03 (logging) e hoje (pickle)

```
import math                # o módulo inteiro, com prefixo
print(math.sqrt(16))
4.0
print(type(math))
<class 'module'>

from math import sqrt, pi  # só o que precisa, sem prefixo
print(sqrt(25), round(pi, 2))
5.0 3.14

import random as rd        # apelido
print(rd.randint(1, 6))
3                          # varia a cada execução
```

Dois erros, duas causas

```
sqrt(16) sem import
NameError: name 'sqrt' is not
defined
```

```
import numpy_fake
ModuleNotFoundError: No module
named 'numpy_fake'
```

Três origens de módulo

1. Biblioteca padrão: vem com o Python (math, random, pickle, logging).
2. Comunidade (PyPI): instala com pip.
3. Seu: qualquer .py na pasta.

Regra de leitura: `math.sqrt` = "a função sqrt que mora no módulo math". O ponto é o mesmo de produto["preço"]: um lugar, um item dentro.

Seu módulo: validacao.py

import executa o arquivo uma vez · `__name__` diz quem é o "principal"

```
# validacao.py (na mesma pasta)
print("carregando validacao.py")    # só para ver o import

def ler_inteiro_nao_negativo(mensagem):
    ...                               # o código da oficina

if __name__ == "__main__":
    print("teste:", ler_inteiro_nao_negativo("Inteiro: "))

# estoque.py (outro arquivo, mesma pasta)
import validacao
carregando validacao.py
import validacao                    # 2ª vez: nada
print(validacao.__name__, __name__)
validacao.__main__
print(validacao.ler_inteiro_nao_negativo("Qtd: "))
```

O que o import faz

Procura `validacao.py` na pasta do script, EXECUTA o arquivo de cima a baixo (por isso o `print` sai) e guarda tudo que foi definido em `validacao`.

if `__name__ == "__main__"`

F5 no `validacao.py` → `__name__` é `"__main__"` → o teste roda.
Importado → `__name__` é `"validacao"` → o teste NÃO roda.

Foi por isso que o `estoque_v2` termina com `if __name__ == "__main__": main()` — dá para importar as funções dele sem abrir o menu.

Mover ler_* para validacao.py e importar

```
# PASSO A: criar validacao.py com as 3 funções da oficina
# ler_texto · ler_inteiro_nao_negativo · ler_preco_nao_negativo
# (recortar do controle_estoque.py, colar no novo arquivo)

# PASSO B: no controle_estoque.py, apagar as 3 e colocar no topo:
from validacao import (ler_texto, ler_inteiro_nao_negativo,
                       ler_preco_nao_negativo)

# PASSO C: rodar o estoque. Tem que funcionar IGUAL.
Código: abc
[!] Entrada inválida: invalid literal for int() with base 10: 'abc'
Código: 101

# PASSO D: no terminal, provar que o módulo é independente
>>> import validacao
>>> [n for n in dir(validacao) if not n.startswith("_")]
['ler_inteiro_nao_negativo', 'ler_preco_nao_negativo', 'ler_texto']
```

Por que vale a pena

Próximo projeto precisa ler um preço? import validacao. Achou bug no ler_preco? Corrige em UM lugar. É o cap. 1 (modularização) em escala de arquivo.

Critério de conclusão

- ✓ controle_estoque.py sem nenhum def ler_*
- ✓ Cadastro recusa abc e -5 como antes.
- ✓ dir(validacao) mostra as 3.

Armadilhas de nome (executadas)

O Python procura módulos primeiro na SUA pasta — e isso tem consequências

1. Batizou o arquivo de random.py

```
import random
random.randint(1, 6)
AttributeError: module 'random' has no
attribute 'randint'
```

O Python achou o SEU random.py antes do oficial.

2. Hífen ou acento no nome

```
import meu-modulo
SyntaxError: invalid syntax

import validação → ModuleNotFoundError
```

Nome de módulo = nome de variável: letras, números, _

3. Apareceu uma pasta __pycache__

É o Python guardando o módulo já traduzido (.pyc) para importar mais rápido da próxima vez. Pode apagar, ele recria. Não vai para o PDF do PTI.

4. Editou o módulo e não mudou nada

No terminal interativo o import só roda uma vez. Alterou validacao.py? Feche e abra o terminal (ou rode o arquivo com F5 de novo).

Ordem de busca (sys.path): 1º a pasta do script, 2º a biblioteca padrão, 3º o que o pip instalou. Por isso o seu random.py ganha.

pip: instalar módulos da comunidade

PyPI (pypi.org) tem centenas de milhares de pacotes. pip é o instalador que vem com o Python.

Comando (no TERMINAL, não no Python)	Faz o quê
<code>python -m pip --version</code>	confirma que o pip está ligado ao Python certo
<code>python -m pip install requests</code>	baixa e instala o pacote (e as dependências dele)
<code>python -m pip install requests==2.32.3</code>	instala uma versão específica
<code>python -m pip list</code>	lista tudo que está instalado
<code>python -m pip show requests</code>	versão, autor, onde foi instalado
<code>python -m pip uninstall requests</code>	remove
<code>python -m pip freeze > requirements.txt</code>	grava a lista de pacotes + versões num arquivo
<code>python -m pip install -r requirements.txt</code>	instala tudo que está no arquivo (em outra máquina)

Por que python -m pip e não só pip? Porque garante que instala no MESMO Python que o VS Code usa. Quem tem 2 Pythons instalados sofre com isso.

No Windows, se python não for reconhecido, use py -m pip. Sem internet, o pip não funciona — e no laboratório pode haver proxy.

Ambiente virtual: uma caixa por projeto

Projeto A precisa da versão 1, projeto B da versão 2. Sem venv, um quebra o outro.

```
# 1) criar (uma vez, dentro da pasta do projeto)
python -m venv venv
```

```
# 2) ativar (toda vez que abrir o terminal)
venv\Scripts\activate          # Windows
source venv/bin/activate       # Linux / macOS
(venv) C:\projeto>             # o prompt muda
```

```
# 3) instalar DENTRO da caixa
python -m pip install requests
```

```
# 4) sair
deactivate
```

O que a pasta venv/ contém

Uma cópia leve do Python + um pip só dele + a pasta lib/ onde os pacotes deste projeto ficam. Apagou a pasta, apagou o ambiente. Nunca vai para o PDF nem para o git.

Windows: "execução de scripts desabilitada"

Erro do PowerShell ao ativar.
Solução: usar o terminal cmd, ou rodar uma vez:
Set-ExecutionPolicy -Scope
CurrentUser RemoteSigned

O VS Code detecta o venv e pergunta se quer usá-lo como interpretador — diga sim. Slide cortável: fica no roteiro.

Referência rápida + estoque v2

Cole na parede do VS Code

Sintaxe	Faz o quê	Devolve
<code>with open("f.txt", "w", encoding="utf-8") as a:</code>	abre para escrever (apaga!) e fecha sozinho	arquivo
<code>a.write("texto\n")</code>	grava a str (só str; \n é seu)	nº de caracteres
<code>a.read() / a.readline() / a.readlines()</code>	tudo / uma linha / lista de linhas	str / str / list
<code>for linha in a: linha.strip()</code>	percorre linha a linha, sem o \n	str
<code>open(f, "rb") / open(f, "wb")</code>	binário: lê/escreve bytes, sem encoding	bytes
<code>pickle.dump(obj, a) / pickle.load(a)</code>	objeto → arquivo / arquivo → objeto	None / objeto
<code>import m · from m import f · import m as x</code>	carrega o módulo (1 vez), traz f, apelida	module
<code>if __name__ == "__main__":</code>	roda só quando o arquivo é o principal	—
<code>python -m pip install pacote</code>	instala da comunidade (terminal)	—

estoque_v2.py (roteiro): carregar_estoque no main, salvar_estoque após cada cadastro, from validacao import ..., opção 3 exporta relatorio.txt. Não é o PTI.

Hoje você aprendeu a

Lembrar depois de fechar. Reaproveitar sem copiar.

Arquivos para o lembrar. Módulos para o reaproveitar. Os dois juntos são o `estoque_v2` — e o primeiro projeto de vocês que sobrevive ao fechar da janela.

Desafio para casa: na prática 1, proteja o `float(nota)` com `try/except ValueError` — linha inválida é ignorada com aviso, e a média usa só as válidas. Teste com `"Dani;abc"`.

Referências: Práticas de Programação, cap. 7 e 8, Editora Senac SP · Documentação oficial: docs.python.org/3/tutorial/inputoutput.html · docs.python.org/3/tutorial/modules.html